

Backend Engineering Intern – Case Study – Solution

Name: Mansi Parmar

Email id: mansiparmar663@gmail.com

Contact: +91 8401599559

Part 1: Code Review & Debugging

Problems Identified

1. SKU Uniqueness Not Enforced

- **Issue:** No check for existing SKU.
- **Impact:** Duplicate SKUs cause conflicts in ordering, reporting, and integrations.
- **Fix:** Validate SKU uniqueness before creating and DB unique constraint.

2. Incorrect Product – Warehouse Relationship

- **Issue:** warehouse_id stored in Product.
- **Impact:** Products cannot exist in multiple warehouses.
- **Fix:** Remove warehouse_id from Product, manage via Inventory table.

3. No Transaction Handling

- **Issue:** Two separate commits.
- **Impact:** Product may be created without inventory if second commit fails.
- **Fix:** Use a single database transaction.

4. No Input Validation

- **Issue:** Assumes all fields exist and are valid.
- **Impact:** Runtime errors, negative quantity, invalid price.
- **Fix:** Validate request payload.

5. Price Precision Issue

- **Issue:** Price treated as float.
- **Impact:** Floating – point inaccuracies.
- **Fix:** Use Decimal with proper DB column type.

6. Optional Fields Not Handled

- **Issue:** Missing support for optional fields.
- **Impact:** API breaks when optional fields are omitted.
- **Fix:** Use `.get()` with defaults.

Corrected Version

```

from decimal import Decimal
from sqlalchemy.exc import IntegrityError

@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    required_fields = ['name', 'sku', 'price', 'warehouse_id']
    for field in required_fields:
        if field not in data:
            return {"error": f"{field} is required"}, 400

    try:
        with db.session.begin():
            product = Product(
                name=data['name'],
                sku=data['sku'],
                price=Decimal(str(data['price']))
            )
            db.session.add(product)

            inventory = Inventory(
                product=product,
                warehouse_id=data['warehouse_id'],
                quantity=data.get('initial_quantity', 0)
            )
            db.session.add(inventory)

```

```
return {"message": "Product created", "product_id": product.id}, 201
```

```
except IntegrityError:
```

```
db.session.rollback()
```

```
return {"error": "SKU already exists"}, 409
```

Part 2: Database Design

Proposed Schema (PostgreSQL)

```
Company(
```

```
    id UUID PK,
```

```
    name TEXT NOT NULL
```

```
);
```

```
Warehouse(
```

```
    id UUID PK,
```

```
    company_id UUID FK REFERENCES Company(id),
```

```
    name TEXT NOT NULL
```

```
);
```

```
Product(
```

```
    id UUID PK,
```

```
    company_id UUID FK REFERENCES Company(id),
```

```
    name TEXT NOT NULL,
```

```
    sku TEXT UNIQUE NOT NULL,
```

```
    price NUMERIC(10,2) NOT NULL,
```

```
    product_type TEXT
```

```
);
```

```
Inventory(
```

```
    id UUID PK,
```

```
    product_id UUID FK REFERENCES Product(id),
```

```

        warehouse_id UUID FK REFERENCES Warehouse(id),
        quantity INT NOT NULL,
        UNIQUE(product_id, warehouse_id)
    );

    Supplier(
        id UUID PK,
        name TEXT,
        contact_email TEXT
    );

    ProductSupplier(
        product_id UUID FK REFERENCES Product(id),
        supplier_id UUID FK REFERENCES Supplier(id),
        PRIMARY KEY(product_id, supplier_id)
    );

    ProductBundle(
        parent_product_id UUID,
        child_product_id UUID,
        quantity INT,
        PRIMARY KEY(parent_product_id, child_product_id)
    );

```

Missing Requirement Questions

- Can SKUs be reused across companies?
- Can products have multiple suppliers with priorities?
- Are bundles sellable independently?
- How is “recent sales activity” defined?
- Should inventory thresholds be configurable per warehouse?

Design Justification

- **Inventory as junction table:** Supports multi – warehouse products.
- **History table:** Enables audit and analytics.
- **Composite indexes:** Fast warehouse – product lookups.

- **UUIDs:** Better scalability in distributed systems.
-

Part 3: API Implementation

Implementation (Python)

```
@app.route('/api/companies/<company_id>/alerts/low-stock', methods=['GET'])
```

```
def low_stock_alerts(company_id):
```

```
    alerts = []
```

```
    inventories = db.session.query(Inventory).join(Warehouse).filter(
```

```
        Warehouse.company_id == company_id
```

```
    ).all()
```

```
    for inv in inventories:
```

```
        threshold = get_threshold(inv.product.product_type)
```

```
        recent_sales = db.session.query(Sales).filter(
```

```
            Sales.product_id == inv.product_id,
```

```
            Sales.created_at >= datetime.utcnow() - timedelta(days=30)
```

```
        ).count()
```

```
        if inv.quantity < threshold and recent_sales > 0:
```

```
            supplier = inv.product.suppliers[0] if inv.product.suppliers else None
```

```
            alerts.append({
```

```
                "product_id": inv.product_id,
```

```
                "product_name": inv.product.name,
```

```
                "sku": inv.product.sku,
```

```

"warehouse_id": inv.warehouse_id,
"warehouse_name": inv.warehouse.name,
"current_stock": inv.quantity,
"threshold": threshold,
"days_until_stockout": estimate_days(inv.product_id),
"supplier": {
    "id": supplier.id,
    "name": supplier.name,
    "contact_email": supplier.contact_email
} if supplier else None
})

return {
    "alerts": alerts,
    "total_alerts": len(alerts)
}

```

Assumptions

- Sales table tracks recent orders.
- Threshold stored in Product.product_type.
- Recent sales = last 30 days.

Edge Cases Considered

- Products without suppliers.
- Zero recent sales.
- Multiple warehouses.
- Missing thresholds.
- Deleted / inactive products.